

Divide and Conquer

Divide and Conquer



Divide and Conquer



Divide and Conquer is a strategy that solves a problem by:

Divide and Conquer



Divide and Conquer is a strategy that solves a problem by:

- 1 Breaking the problem into subproblems that are themselves smaller instances of the same type of problem.

Divide and Conquer

Divide and Conquer is a strategy that solves a problem by:

- 1 Breaking the problem into subproblems that are themselves smaller instances of the same type of problem.
- 2 Recursively solving these subproblems.

Divide and Conquer

Divide and Conquer is a strategy that solves a problem by:

- 1 Breaking the problem into subproblems that are themselves smaller instances of the same type of problem.
- 2 Recursively solving these subproblems.
- 3 Appropriately combining their answers.

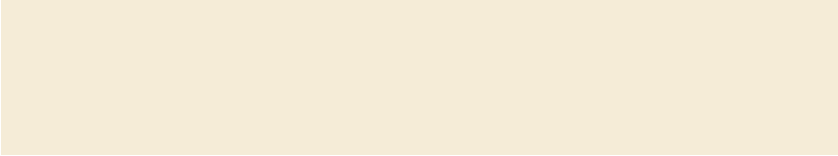
Searching



Searching



Search



Searching

Search

Input: A list of numbers, sorted in increasing order,
 $A = [a_0, a_1, a_2, a_3, \dots, a_{n-1}]$ and a number x .

Searching

Search

Input: A list of numbers, sorted in increasing order,
 $A = [a_0, a_1, a_2, a_3, \dots, a_{n-1}]$ and a number x .

Goal: Return a position of x in A if $x \in A$ or return a statement that x is not in the list.

Searching

Search

Input: A list of numbers, sorted in increasing order,
 $A = [a_0, a_1, a_2, a_3, \dots, a_{n-1}]$ and a number x .

Goal: Return a position of x in A if $x \in A$ or return a statement that x is not in the list.

- How would a four year old do this?

Searching

Search

Input: A list of numbers, sorted in increasing order,
 $A = [a_0, a_1, a_2, a_3, \dots, a_{n-1}]$ and a number x .

Goal: Return a position of x in A if $x \in A$ or return a statement that x is not in the list.

- How would a four year old do this?
- How would a 2nd grader do this?

Searching

Search

Input: A list of numbers, sorted in increasing order,
 $A = [a_0, a_1, a_2, a_3, \dots, a_{n-1}]$ and a number x .

Goal: Return a position of x in A if $x \in A$ or return a statement that x is not in the list.

- How would a four year old do this?
- How would a 2nd grader do this?
- How would you do it?

BinarySearch



BinarySearch

BinarySearch(A , x , min, max)

BinarySearch

BinarySearch(A , x , min, max)

Input: An array of n sorted numbers A , number x , integer indices min, and max.

BinarySearch

BinarySearch(A , x , min , max)

Input: An array of n sorted numbers A , number x , integer indices min , and max . (Initially $\text{min} = 0$ and $\text{max} = n - 1$)

BinarySearch

BinarySearch($A, x, \text{min}, \text{max}$)

Input: An array of n sorted numbers A , number x , integer indices min , and max . (Initially $\text{min} = 0$ and $\text{max} = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

BinarySearch

BinarySearch($A, x, \text{min}, \text{max}$)

Input: An array of n sorted numbers A , number x , integer indices min , and max . (Initially $\text{min} = 0$ and $\text{max} = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\text{max} < \text{min}$ **then**

BinarySearch

BinarySearch($A, x, \text{min}, \text{max}$)

Input: An array of n sorted numbers A , number x , integer indices min , and max . (Initially $\text{min} = 0$ and $\text{max} = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\text{max} < \text{min}$ **then**
 return -1

BinarySearch

BinarySearch($A, x, \text{min}, \text{max}$)

Input: An array of n sorted numbers A , number x , integer indices min , and max . (Initially $\text{min} = 0$ and $\text{max} = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\text{max} < \text{min}$ **then**

return -1

2: **else**

BinarySearch

BinarySearch($A, x, \min, \max$)

Input: An array of n sorted numbers A , number x , integer indices $\min$, and $\max$. (Initially $\min = 0$ and $\max = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\max < \min$ **then**

return -1

2: **else**

3: **if** $x < A[\lfloor (\max + \min)/2 \rfloor]$ **then**

BinarySearch

BinarySearch($A, x, \min, \max$)

Input: An array of n sorted numbers A , number x , integer indices $\min$, and $\max$. (Initially $\min = 0$ and $\max = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\max < \min$ **then**

return -1

2: **else**

3: **if** $x < A[\lfloor (\max + \min)/2 \rfloor]$ **then**

return BinarySearch($A, x, \min, \lfloor (\max + \min)/2 \rfloor - 1$)

BinarySearch

BinarySearch($A, x, \min, \max$)

Input: An array of n sorted numbers A , number x , integer indices $\min$, and $\max$. (Initially $\min = 0$ and $\max = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\max < \min$ **then**

return -1

2: **else**

3: **if** $x < A[\lfloor (\max + \min)/2 \rfloor]$ **then**

return BinarySearch($A, x, \min, \lfloor (\max + \min)/2 \rfloor - 1$)

4: **else if** $x > A[\lfloor (\max + \min)/2 \rfloor]$ **then**

BinarySearch

BinarySearch($A, x, \min, \max$)

Input: An array of n sorted numbers A , number x , integer indices $\min$, and $\max$. (Initially $\min = 0$ and $\max = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\max < \min$ **then**

return -1

2: **else**

3: **if** $x < A[\lfloor (\max + \min)/2 \rfloor]$ **then**

return BinarySearch($A, x, \min, \lfloor (\max + \min)/2 \rfloor - 1$)

4: **else if** $x > A[\lfloor (\max + \min)/2 \rfloor]$ **then**

return BinarySearch($A, x, \lfloor (\max + \min)/2 \rfloor + 1, \max$)

BinarySearch

BinarySearch($A, x, \min, \max$)

Input: An array of n sorted numbers A , number x , integer indices $\min$, and $\max$. (Initially $\min = 0$ and $\max = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\max < \min$ **then**

return -1

2: **else**

3: **if** $x < A[\lfloor (\max + \min)/2 \rfloor]$ **then**

return BinarySearch($A, x, \min, \lfloor (\max + \min)/2 \rfloor - 1$)

4: **else if** $x > A[\lfloor (\max + \min)/2 \rfloor]$ **then**

return BinarySearch($A, x, \lfloor (\max + \min)/2 \rfloor + 1, \max$)

5: **else**

BinarySearch

BinarySearch($A, x, \min, \max$)

Input: An array of n sorted numbers A , number x , integer indices $\min$, and $\max$. (Initially $\min = 0$ and $\max = n - 1$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

1: **if** $\max < \min$ **then**

return -1

2: **else**

3: **if** $x < A[\lfloor (\max + \min)/2 \rfloor]$ **then**

return BinarySearch($A, x, \min, \lfloor (\max + \min)/2 \rfloor - 1$)

4: **else if** $x > A[\lfloor (\max + \min)/2 \rfloor]$ **then**

return BinarySearch($A, x, \lfloor (\max + \min)/2 \rfloor + 1, \max$)

5: **else**

return $\lfloor (\max + \min)/2 \rfloor$

BinarySearch



BinarySearch



We must ask ourselves two questions about this algorithm:

BinarySearch



We must ask ourselves two questions about this algorithm:

- Is it correct?

BinarySearch



We must ask ourselves two questions about this algorithm:

- Is it correct?
- What is the running time?

BinarySearch

We must ask ourselves two questions about this algorithm:

- Is it correct?
- What is the running time?
 - At each stage we divide the problem in half and it takes constant time to “combine” solutions:

BinarySearch

We must ask ourselves two questions about this algorithm:

- Is it correct?
- What is the running time?
 - ▣ At each stage we divide the problem in half and it takes constant time to “combine” solutions:
 - ▣ $T(n) = T(n/2) + O(1)$

BinarySearch - Correctness



BinarySearch - Correctness

Is it correct?

BinarySearch - Correctness

Is it correct?

Lemma

If x is an element in the list, BinarySearch will return an index i such that $A[i] = x$.

BinarySearch - Correctness

Is it correct?

Lemma

If x is an element in the list, BinarySearch will return an index i such that $A[i] = x$.

Proof.

By induction, in class.



BinarySearch - Correctness

Is it correct?

Lemma

If x is an element in the list, BinarySearch will return an index i such that $A[i] = x$.

Proof.

By induction, in class.



Lemma

If x is not an element of the list, BinarySearch will return -1 .

Recurrence Relations



Recurrence Relations



You have two main choices when it comes to solving recurrence relations:

Recurrence Relations

You have two main choices when it comes to solving recurrence relations:

- The tree method (my favorite)

Recurrence Relations

You have two main choices when it comes to solving recurrence relations:

- The tree method (my favorite)
- The Master Theorem

Recurrence Relations

You have two main choices when it comes to solving recurrence relations:

- The tree method (my favorite)
- The Master Theorem
 - If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for $a > 0, b > 1, d \geq 0$ then:

Recurrence Relations

You have two main choices when it comes to solving recurrence relations:

- The tree method (my favorite)
- The Master Theorem
 - If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for $a > 0, b > 1, d \geq 0$ then:
 - $T(n) = O(n^d)$ if $d > \log_b a$

Recurrence Relations

You have two main choices when it comes to solving recurrence relations:

- The tree method (my favorite)
- The Master Theorem
 - If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for $a > 0, b > 1, d \geq 0$ then:
 - $T(n) = O(n^d)$ if $d > \log_b a$
 - $T(n) = O(n^d \log n)$ if $d = \log_b a$

Recurrence Relations

You have two main choices when it comes to solving recurrence relations:

- The tree method (my favorite)

- The Master Theorem

- If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for $a > 0, b > 1, d \geq 0$ then:

- $T(n) = O(n^d)$ if $d > \log_b a$

- $T(n) = O(n^d \log n)$ if $d = \log_b a$

- $T(n) = O(n^{\log_b a})$ if $d < \log_b a$

Recurrence Relations - Binary Search

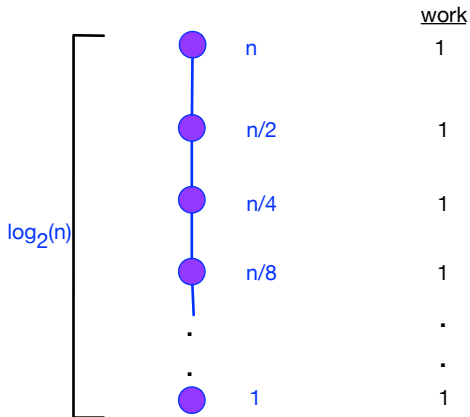


Recurrence Relations - Binary Search

$$T(n) = T(n/2) + O(1)$$

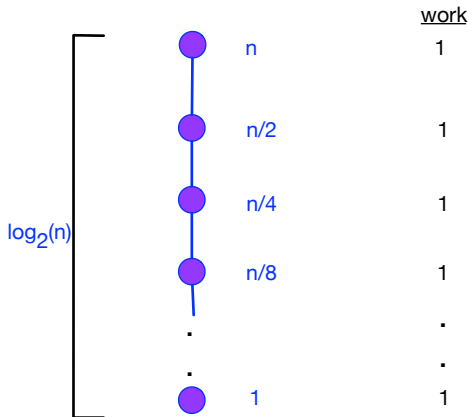
Recurrence Relations - Binary Search

$$T(n) = T(n/2) + O(1)$$



Recurrence Relations - Binary Search

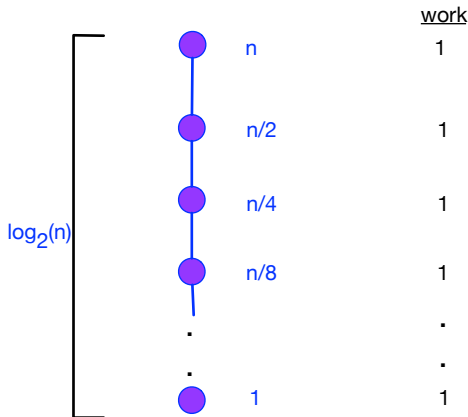
$$T(n) = T(n/2) + O(1)$$



$$= O(1 + 1 + \dots + 1)$$

Recurrence Relations - Binary Search

$$T(n) = T(n/2) + O(1)$$



$$= O(1 + 1 + \cdots + 1) = O(\log_2(n))$$

Recurrence Relations - Merge Sort

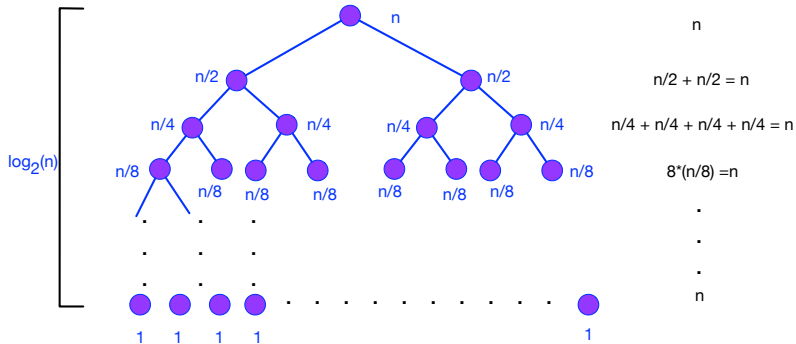


Recurrence Relations - Merge Sort

$$T(n) = 2T(n/2) + O(n)$$

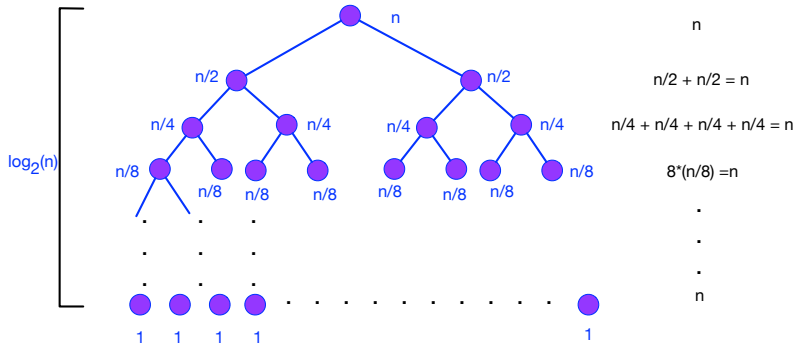
Recurrence Relations - Merge Sort

$$T(n) = 2T(n/2) + O(n)$$



Recurrence Relations - Merge Sort

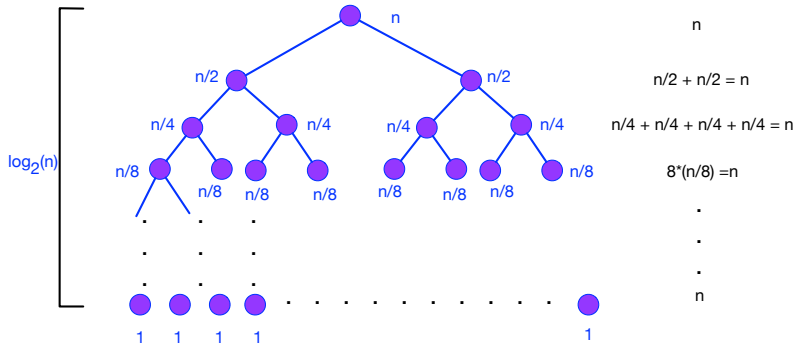
$$T(n) = 2T(n/2) + O(n)$$



$$= O(n + n + n + \cdots + n)$$

Recurrence Relations - Merge Sort

$$T(n) = 2T(n/2) + O(n)$$



$$= O(n + n + n + \cdots + n) = O(n \log_2(n))$$

Recurrence Relations - More Practice

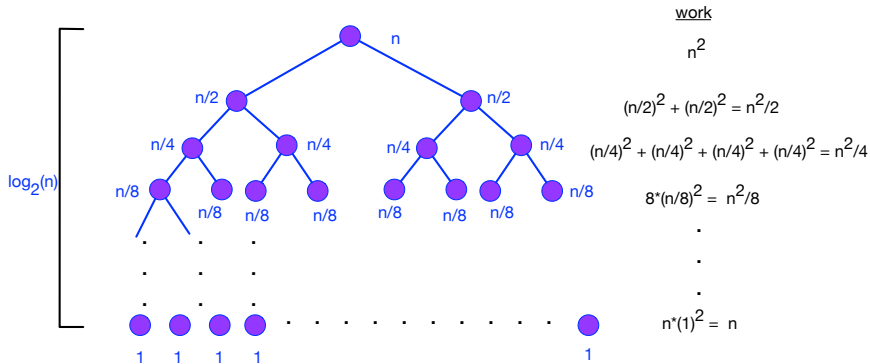


Recurrence Relations - More Practice

$$T(n) = 2T(n/2) + O(n^2)$$

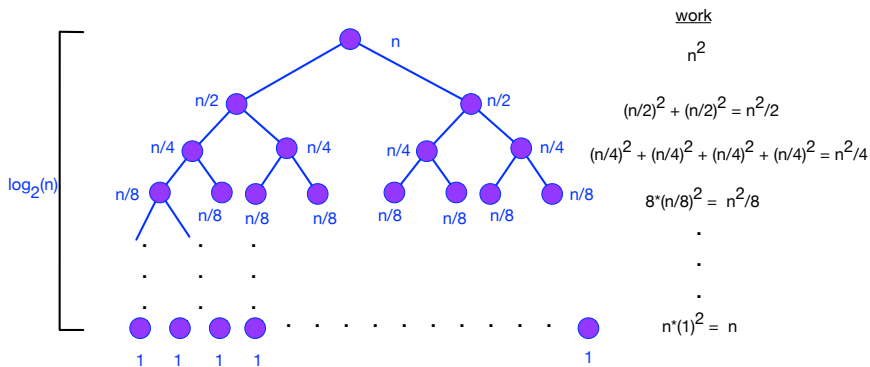
Recurrence Relations - More Practice

$$T(n) = 2T(n/2) + O(n^2)$$



 University of Twente

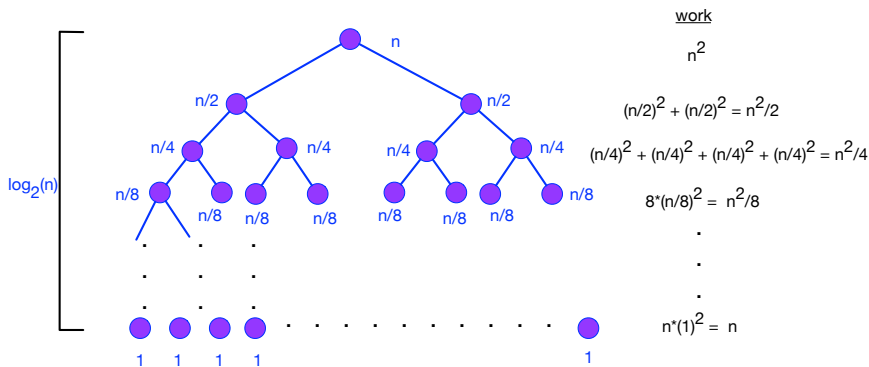
$$T(n) = 2T(n/2) + O(n^2)$$



$$= O(n^2 + n^2/2 + n^2/4 + n^2/8 \dots + n^2/n)$$

Recurrence Relations - More Practice

$$T(n) = 2T(n/2) + O(n^2)$$



$$= O(n^2 + n^2/2 + n^2/4 + n^2/8 \cdots + n^2/n) \leq O(2n^2) = O(n^2)$$

Recurrence Relations - More Practice

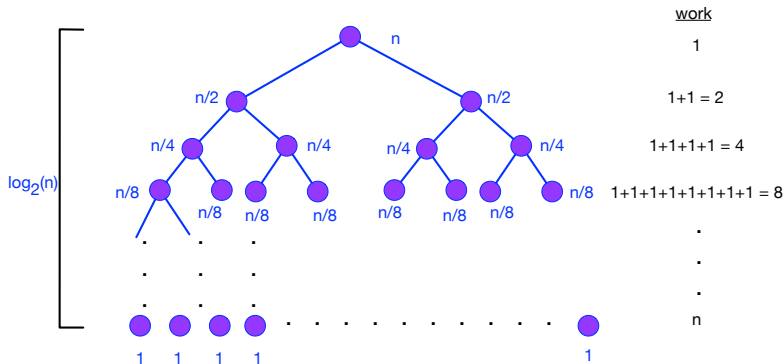


Recurrence Relations - More Practice

$$T(n) = 2T(n/2) + O(1)$$

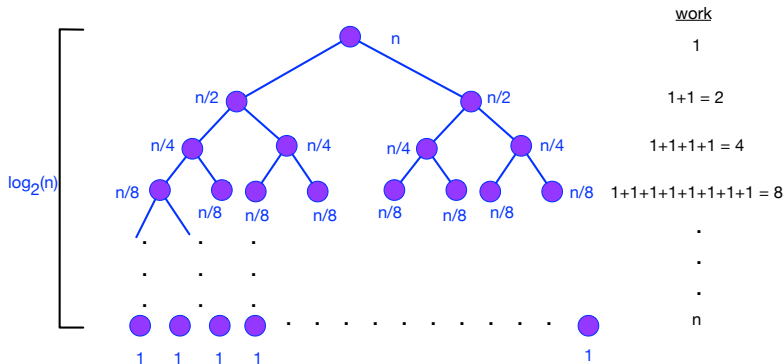
Recurrence Relations - More Practice

$$T(n) = 2T(n/2) + O(1)$$



Recurrence Relations - More Practice

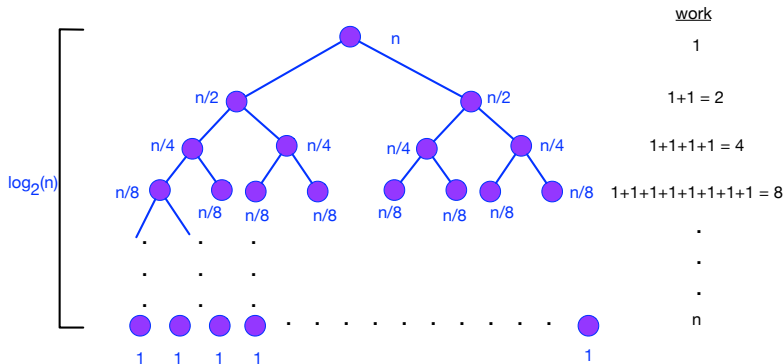
$$T(n) = 2T(n/2) + O(1)$$



$$= O(1 + 2 + 4 + \dots + n)$$

Recurrence Relations - More Practice

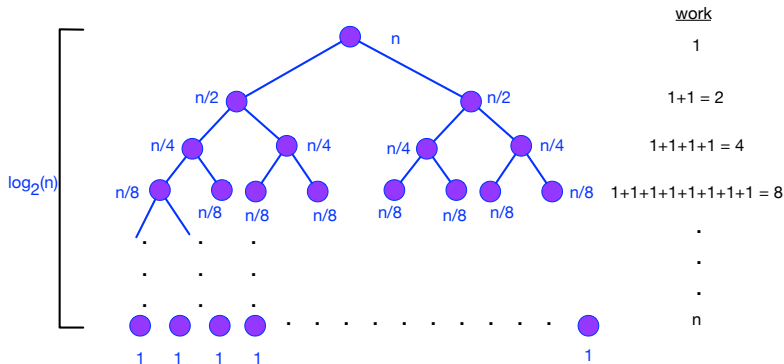
$$T(n) = 2T(n/2) + O(1)$$



$$= O(1 + 2 + 4 + \cdots + n) = O(n + n/2 + n/4 + n/8 + \cdots + n/n)$$

Recurrence Relations - More Practice

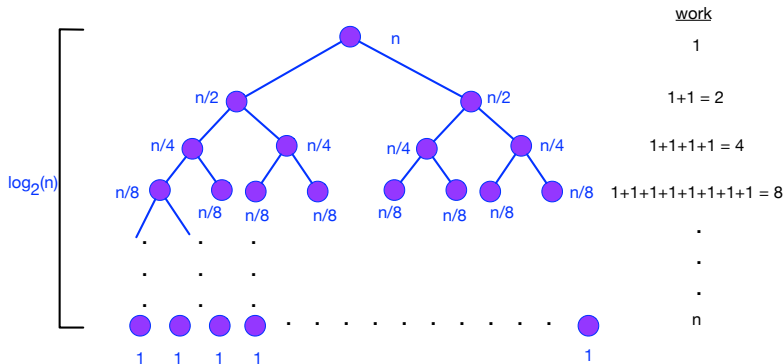
$$T(n) = 2T(n/2) + O(1)$$



$$= O(1 + 2 + 4 + \dots + n) = O(n + n/2 + n/4 + n/8 + \dots + n/n) \leq O(2n)$$

Recurrence Relations - More Practice

$$T(n) = 2T(n/2) + O(1)$$



$$\begin{aligned} &= O(1 + 2 + 4 + \cdots + n) = O(n + n/2 + n/4 + n/8 + \cdots + n/n) \leq \\ &O(2n) = O(n) \end{aligned}$$

Recurrence Relations - Reduce by One

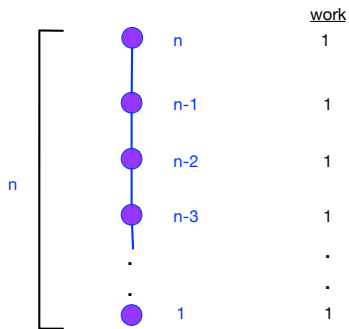


Recurrence Relations - Reduce by One

$$T(n) = T(n - 1) + O(1)$$

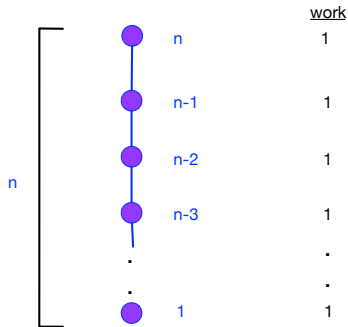
Recurrence Relations - Reduce by One

$$T(n) = T(n-1) + O(1)$$



Recurrence Relations - Reduce by One

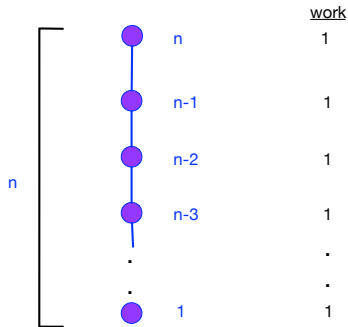
$$T(n) = T(n-1) + O(1)$$



$$= O(1 + 1 + 1 + \cdots + 1)$$

Recurrence Relations - Reduce by One

$$T(n) = T(n-1) + O(1)$$



$$= O(1 + 1 + 1 + \cdots + 1) = O(n)$$

Recurrence Relations - Reduce by One

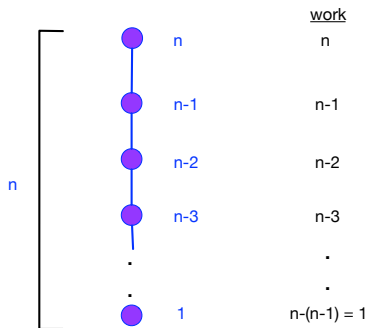


Recurrence Relations - Reduce by One

$$T(n) = T(n - 1) + O(n)$$

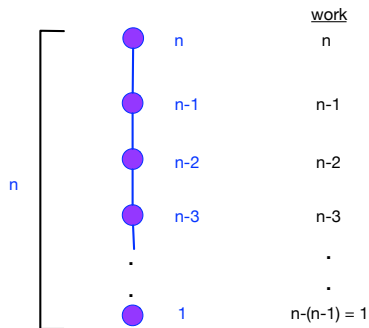
Recurrence Relations - Reduce by One

$$T(n) = T(n-1) + O(n)$$



Recurrence Relations - Reduce by One

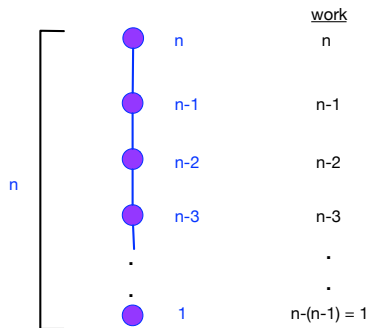
$$T(n) = T(n-1) + O(n)$$



$$= O(n + (n-1) + (n-2) + (n-3) + \dots + (n - (n-2)) + (n - (n-1)))$$

Recurrence Relations - Reduce by One

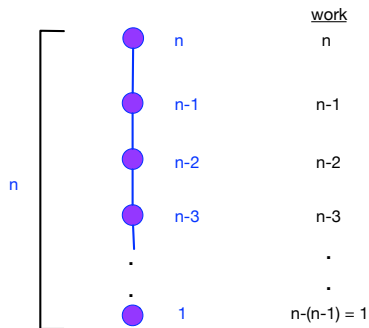
$$T(n) = T(n-1) + O(n)$$



$$= O(n + (n-1) + (n-2) + (n-3) + \dots + (n - (n-2)) + (n - (n-1))) \approx O(n^2/2)$$

Recurrence Relations - Reduce by One

$$T(n) = T(n-1) + O(n)$$



$$\begin{aligned} &= O(n + (n-1) + (n-2) + (n-3) + \cdots + (n - (n-2)) + (n - (n-1))) \approx \\ &O(n^2/2) = O(n^2) \end{aligned}$$

Matrix Multiplication



Matrix Multiplication



Who remembers how to multiply two $n \times n$ matrices together?

Matrix Multiplication

Who remembers how to multiply two $n \times n$ matrices together?

Example:

$$\begin{pmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 2 & 1 & 3 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 5 & 1 & 0 \\ 1 & 3 & 3 \end{pmatrix}$$

Matrix Multiplication

Who remembers how to multiply two $n \times n$ matrices together?

Example:

$$\begin{pmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 2 & 1 & 3 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 5 & 1 & 0 \\ 1 & 3 & 3 \end{pmatrix}$$

- What algorithm did you use?

Matrix Multiplication

Who remembers how to multiply two $n \times n$ matrices together?

Example:

$$\begin{pmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 2 & 1 & 3 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 5 & 1 & 0 \\ 1 & 3 & 3 \end{pmatrix}$$

- What algorithm did you use?
- What is the running time?

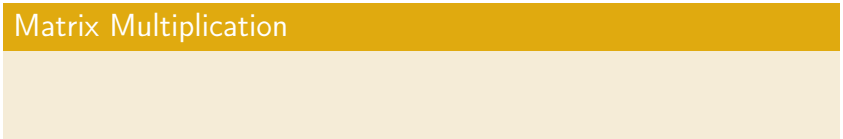
Matrix Multiplication



Matrix Multiplication



Matrix Multiplication



Matrix Multiplication

Matrix Multiplication

Input: Two $n \times n$ matrices, X and Y .

Matrix Multiplication

Matrix Multiplication

Input: Two $n \times n$ matrices, X and Y .

Goal: Return the product XY .

Matrix Multiplication

Matrix Multiplication

Input: Two $n \times n$ matrices, X and Y .

Goal: Return the product XY .

To simplify analysis, suppose that n is a power of 2.

Matrix Multiplication



Matrix Multiplication

MMult(X,Y)

Matrix Multiplication

MMult(X, Y)

Input: Two $n \times n$ matrices, X and Y , (where n is a power of 2)

Matrix Multiplication

MMult(X, Y)

Input: Two $n \times n$ matrices, X and Y , (where n is a power of 2)

Output: The product XY

Matrix Multiplication

MMult(X, Y)

Input: Two $n \times n$ matrices, X and Y , (where n is a power of 2)

Output: The product XY

1: if $n = 1$

Matrix Multiplication

MMult(X, Y)

Input: Two $n \times n$ matrices, X and Y , (where n is a power of 2)

Output: The product XY

1: **if** $n = 1$ (i.e. $X = (x)$, $Y = (y)$) **then**

Matrix Multiplication

MMult(X, Y)

Input: Two $n \times n$ matrices, X and Y , (where n is a power of 2)

Output: The product XY

1: **if** $n = 1$ (i.e. $X = (x)$, $Y = (y)$) **then**
 return ($x \times y$)

Matrix Multiplication

MMult(X, Y)

Input: Two $n \times n$ matrices, X and Y , (where n is a power of 2)

Output: The product XY

- 1: **if** $n = 1$ (i.e. $X = (x)$, $Y = (y)$) **then**
 return ($x \times y$)
- 2: **else** Decompose X and Y into four $n/2 \times n/2$ blocks each:

Matrix Multiplication

MMult(X, Y)

Input: Two $n \times n$ matrices, X and Y , (where n is a power of 2)

Output: The product XY

- 1: **if** $n = 1$ (i.e. $X = (x)$, $Y = (y)$) **then**
 return $(x \times y)$
- 2: **else** Decompose X and Y into four $n/2 \times n/2$ blocks each:

$$X = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, Y = \begin{pmatrix} E & F \\ G & H \end{pmatrix}$$

Matrix Multiplication

MMult(X, Y)

Input: Two $n \times n$ matrices, X and Y , (where n is a power of 2)

Output: The product XY

1: **if** $n = 1$ (i.e. $X = (x)$, $Y = (y)$) **then**

return $(x \times y)$

2: **else** Decompose X and Y into four $n/2 \times n/2$ blocks each:

$$X = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, Y = \begin{pmatrix} E & F \\ G & H \end{pmatrix}$$

return $\begin{pmatrix} \text{MMult}(A, E) + \text{MMult}(B, G) & \text{MMult}(A, F) + \text{MMult}(B, H) \\ \text{MMult}(C, E) + \text{MMult}(D, G) & \text{MMult}(C, F) + \text{MMult}(D, H) \end{pmatrix}$

Matrix Multiplication



Matrix Multiplication

- Is the algorithm correct?

Matrix Multiplication

- Is the algorithm correct?
- What is the running time?

Matrix Multiplication

- Is the algorithm correct?
- What is the running time?
 - ▣ $T(n) = 8T(n/2) + O(n^2)$

Matrix Multiplication

- Is the algorithm correct?
- What is the running time?
 - ▣ $T(n) = 8T(n/2) + O(n^2) = O(n^3)$

Matrix Multiplication

- Is the algorithm correct?
- What is the running time?
 - ▣ $T(n) = 8T(n/2) + O(n^2) = O(n^3)$
 - ▣ There was no improvement from the linear algebra method.

Matrix Multiplication

- Is the algorithm correct?
- What is the running time?
 - ▣ $T(n) = 8T(n/2) + O(n^2) = O(n^3)$
 - ▣ There was no improvement from the linear algebra method.
- There is a way to only perform 7 multiplications (Strassen's method).

Matrix Multiplication

- Is the algorithm correct?
- What is the running time?
 - $T(n) = 8T(n/2) + O(n^2) = O(n^3)$
 - There was no improvement from the linear algebra method.
- There is a way to only perform 7 multiplications (Strassen's method).

$$XY = \begin{pmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{pmatrix}$$

Where: $P_1 = A(F - H)$, $P_2 = (A + B)H$, $P_3 = (C + D)E$,
 $P_4 = D(G - E)$, $P_5 = (A + D)(E + H)$,
 $P_6 = (B - D)(G + H)$, and $P_7 = (A - C)(E + F)$.

- Then $T(n) = 7T(n/2) + O(n^2)$

Matrix Multiplication

- Is the algorithm correct?
- What is the running time?
 - $T(n) = 8T(n/2) + O(n^2) = O(n^3)$
 - There was no improvement from the linear algebra method.
- There is a way to only perform 7 multiplications (Strassen's method).

$$XY = \begin{pmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{pmatrix}$$

Where: $P_1 = A(F - H)$, $P_2 = (A + B)H$, $P_3 = (C + D)E$,
 $P_4 = D(G - E)$, $P_5 = (A + D)(E + H)$,
 $P_6 = (B - D)(G + H)$, and $P_7 = (A - C)(E + F)$.

- Then $T(n) = 7T(n/2) + O(n^2) = O(n^{2.81})$.

Matrix Multiplication

- Is the algorithm correct?
- What is the running time?
 - $T(n) = 8T(n/2) + O(n^2) = O(n^3)$
 - There was no improvement from the linear algebra method.
- There is a way to only perform 7 multiplications (Strassen's method).

$$XY = \begin{pmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{pmatrix}$$

Where: $P_1 = A(F - H)$, $P_2 = (A + B)H$, $P_3 = (C + D)E$,
 $P_4 = D(G - E)$, $P_5 = (A + D)(E + H)$,
 $P_6 = (B - D)(G + H)$, and $P_7 = (A - C)(E + F)$.

■ Then $T(n) = 7T(n/2) + O(n^2) = O(n^{2.81})$.

There exists an $O(n^{2.3727})$ algorithm (Virginia Vassilevska Williams).

Sorting



Sorting



Sorting

Sorting

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Sorting

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Sorting

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Example:

Given: 4, 907, 34, 18, 42, 36, 71, 34, 16

Sorting

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Example:

Given: 4, 907, 34, 18, 42, 36, 71, 34, 16

Return: 4, 16, 18, 34, 34, 36, 42, 71, 907

SelectionSort



SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort($A[0, \dots, n - 1]$)

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list sorted in increasing order

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list sorted in increasing order

1: **for** $i = 0, \dots, n - 1$ **do**

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list sorted in increasing order

- 1: **for** $i = 0, \dots, n - 1$ **do**
- 2: Find min of $A[i, \dots, n - 1]$.

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort($A[0, \dots, n-1]$)

Input: A list of unsorted numbers $A[0, \dots, n-1]$

Output: The same list sorted in increasing order

- 1: **for** $i = 0, \dots, n-1$ **do**
- 2: Find min of $A[i, \dots, n-1]$.
- 3: Suppose that the min occurs at position j .

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort($A[0, \dots, n-1]$)

Input: A list of unsorted numbers $A[0, \dots, n-1]$

Output: The same list sorted in increasing order

- 1: **for** $i = 0, \dots, n-1$ **do**
 - 2: Find min of $A[i, \dots, n-1]$.
 - 3: Suppose that the min occurs at position j .
 - 4: Swap $A[i]$ with $A[j]$.
-

SelectionSort - Correctness



SelectionSort - Correctness

Lemma

Upon completion of SelectionSort, for any $i \in \{1, \dots, n - 1\}$, $A[i - 1] \leq A[i]$. Furthermore, all elements of the input list are in A .

SelectionSort - Correctness

Lemma

Upon completion of SelectionSort, for any $i \in \{1, \dots, n - 1\}$, $A[i - 1] \leq A[i]$. Furthermore, all elements of the input list are in A .

Proof.

Suppose, for a contradiction, that there is a j such that upon completion of SelectionSort, $A[j - 1] > A[j]$.

SelectionSort - Correctness

Lemma

Upon completion of SelectionSort, for any $i \in \{1, \dots, n-1\}$, $A[i-1] \leq A[i]$. Furthermore, all elements of the input list are in A .

Proof.

Suppose, for a contradiction, that there is a j such that upon completion of SelectionSort, $A[j-1] > A[j]$.

Let $A[j-1] = x$ and $A[j] = y$.

SelectionSort - Correctness

Lemma

Upon completion of SelectionSort, for any $i \in \{1, \dots, n-1\}$, $A[i-1] \leq A[i]$. Furthermore, all elements of the input list are in A .

Proof.

Suppose, for a contradiction, that there is a j such that upon completion of SelectionSort, $A[j-1] > A[j]$.

Let $A[j-1] = x$ and $A[j] = y$.

At iteration $j-1$ of the algorithm, $A[j-1]$ was set to be x (step 4).

SelectionSort - Correctness

Lemma

Upon completion of SelectionSort, for any $i \in \{1, \dots, n - 1\}$, $A[i - 1] \leq A[i]$. Furthermore, all elements of the input list are in A .

Proof.

Suppose, for a contradiction, that there is a j such that upon completion of SelectionSort, $A[j - 1] > A[j]$.

Let $A[j - 1] = x$ and $A[j] = y$.

At iteration $j - 1$ of the algorithm, $A[j - 1]$ was set to be x (step 4). Thus, at iteration $j - 1$, x must have been the smallest remaining element (step 2).

SelectionSort - Correctness

Lemma

Upon completion of SelectionSort, for any $i \in \{1, \dots, n - 1\}$, $A[i - 1] \leq A[i]$. Furthermore, all elements of the input list are in A .

Proof.

Suppose, for a contradiction, that there is a j such that upon completion of SelectionSort, $A[j - 1] > A[j]$.

Let $A[j - 1] = x$ and $A[j] = y$.

At iteration $j - 1$ of the algorithm, $A[j - 1]$ was set to be x (step 4). Thus, at iteration $j - 1$, x must have been the smallest remaining element (step 2).

Contradiction.

SelectionSort - Correctness

Lemma

Upon completion of SelectionSort, for any $i \in \{1, \dots, n-1\}$, $A[i-1] \leq A[i]$. Furthermore, all elements of the input list are in A .

Proof.

Suppose, for a contradiction, that there is a j such that upon completion of SelectionSort, $A[j-1] > A[j]$.

Let $A[j-1] = x$ and $A[j] = y$.

At iteration $j-1$ of the algorithm, $A[j-1]$ was set to be x (step 4). Thus, at iteration $j-1$, x must have been the smallest remaining element (step 2).

Contradiction. Because y was a remaining element at iteration $j-1$ and $y < x$.



SelectionSort - Correctness

Lemma

Upon completion of SelectionSort, for any $i \in \{1, \dots, n-1\}$, $A[i-1] \leq A[i]$. Furthermore, all elements of the input list are in A .

Proof.

Suppose, for a contradiction, that there is a j such that upon completion of SelectionSort, $A[j-1] > A[j]$.

Let $A[j-1] = x$ and $A[j] = y$.

At iteration $j-1$ of the algorithm, $A[j-1]$ was set to be x (step 4). Thus, at iteration $j-1$, x must have been the smallest remaining element (step 2).

Contradiction. Because y was a remaining element at iteration $j-1$ and $y < x$. □

How can we show that all elements of the input list are in A ?

SelectionSort - Running Time

□ Running Time:

SelectionSort - Running Time

□ Running Time:

□ $T(n) = T(n - 1) + O(n)$

SelectionSort - Running Time

□ Running Time:

■ $T(n) = T(n - 1) + O(n) = O(n^2)$

MergeSort



MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[0, \dots, n - 1]$)

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list, sorted in increasing order

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list, sorted in increasing order

1: **if** $n \leq 1$ **then**

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list, sorted in increasing order

1: **if** $n \leq 1$ **then**
 return A

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list, sorted in increasing order

- 1: **if** $n \leq 1$ **then**
 return A
- 2: **else**

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list, sorted in increasing order

1: **if** $n \leq 1$ **then**

return A

2: **else**

return Merge(MergeSort($A[0, \dots, \lfloor n/2 \rfloor]$), MergeSort($A[\lfloor n/2 \rfloor + 1, \dots, n - 1]$))

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[0, \dots, n - 1]$)

Input: A list of unsorted numbers $A[0, \dots, n - 1]$

Output: The same list, sorted in increasing order

1: **if** $n \leq 1$ **then**

return A

2: **else**

return Merge(MergeSort($A[0, \dots, \lfloor n/2 \rfloor]$), MergeSort($A[\lfloor n/2 \rfloor + 1, \dots, n - 1]$))

MergeSort



MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k - 1], y[0, \dots, \ell - 1]$)

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k-1], y[0, \dots, \ell-1]$)

Input: Two sorted lists, $x[0, \dots, k-1]$ and $y[0, \dots, \ell-1]$

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k - 1], y[0, \dots, \ell - 1]$)

Input: Two sorted lists, $x[0, \dots, k - 1]$ and $y[0, \dots, \ell - 1]$

Output: One sorted list that contains all elements of both lists.

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k-1], y[0, \dots, \ell-1]$)

Input: Two sorted lists, $x[0, \dots, k-1]$ and $y[0, \dots, \ell-1]$

Output: One sorted list that contains all elements of both lists.

1: **if** $x = \emptyset$ **then**

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k-1], y[0, \dots, \ell-1]$)

Input: Two sorted lists, $x[0, \dots, k-1]$ and $y[0, \dots, \ell-1]$

Output: One sorted list that contains all elements of both lists.

1: if $x = \emptyset$ then return y

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k-1], y[0, \dots, \ell-1]$)

Input: Two sorted lists, $x[0, \dots, k-1]$ and $y[0, \dots, \ell-1]$

Output: One sorted list that contains all elements of both lists.

1: if $x = \emptyset$ then return y

2: if $y = \emptyset$ then

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k-1], y[0, \dots, \ell-1]$)

Input: Two sorted lists, $x[0, \dots, k-1]$ and $y[0, \dots, \ell-1]$

Output: One sorted list that contains all elements of both lists.

1: if $x = \emptyset$ then return y

2: if $y = \emptyset$ then return x

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k-1], y[0, \dots, \ell-1]$)

Input: Two sorted lists, $x[0, \dots, k-1]$ and $y[0, \dots, \ell-1]$

Output: One sorted list that contains all elements of both lists.

- 1: if $x = \emptyset$ then return y
- 2: if $y = \emptyset$ then return x
- 3: if $x[0] \leq y[0]$ then

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k-1], y[0, \dots, \ell-1]$)

Input: Two sorted lists, $x[0, \dots, k-1]$ and $y[0, \dots, \ell-1]$

Output: One sorted list that contains all elements of both lists.

- 1: if $x = \emptyset$ then return y
- 2: if $y = \emptyset$ then return x
- 3: if $x[0] \leq y[0]$ then
 return $x[0] \circ \text{Merge}(x[1, \dots, k-1], y[0, \dots, \ell-1])$

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k - 1], y[0, \dots, \ell - 1]$)

Input: Two sorted lists, $x[0, \dots, k - 1]$ and $y[0, \dots, \ell - 1]$

Output: One sorted list that contains all elements of both lists.

- 1: if $x = \emptyset$ then return y
- 2: if $y = \emptyset$ then return x
- 3: if $x[0] \leq y[0]$ then
 return $x[0] \circ \text{Merge}(x[1, \dots, k - 1], y[0, \dots, \ell - 1])$
- 4: else

MergeSort

Sorting

Input: A sequence of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return a list of the same numbers sorted in increasing order.

Merge($x[0, \dots, k-1], y[0, \dots, \ell-1]$)

Input: Two sorted lists, $x[0, \dots, k-1]$ and $y[0, \dots, \ell-1]$

Output: One sorted list that contains all elements of both lists.

- 1: if $x = \emptyset$ then return y
 - 2: if $y = \emptyset$ then return x
 - 3: if $x[0] \leq y[0]$ then
 return $x[0] \circ \text{Merge}(x[1, \dots, k-1], y[0, \dots, \ell-1])$
 - 4: else
 return $y[0] \circ \text{Merge}(x[0, \dots, k-1], y[1, \dots, \ell-1])$
-

MergeSort - Correctness



MergeSort - Correctness

Theorem

Merge correctly merges two sorted lists.

MergeSort - Correctness

Theorem

Merge correctly merges two sorted lists.

Theorem

Given two sorted lists, x and y , of total size n Merge returns a sorted list containing all elements from x and y .

MergeSort - Correctness

Theorem

Merge correctly merges two sorted lists.

Theorem

Given two sorted lists, x and y , of total size n Merge returns a sorted list containing all elements from x and y .

Proof.

We will proceed by induction on the total size of the lists being merged.

MergeSort - Correctness

Theorem

Merge correctly merges two sorted lists.

Theorem

Given two sorted lists, x and y , of total size n Merge returns a sorted list containing all elements from x and y .

Proof.

We will proceed by induction on the total size of the lists being merged.

□ Base Case: ($n = 1$)

MergeSort - Correctness

Theorem

Merge correctly merges two sorted lists.

Theorem

Given two sorted lists, x and y , of total size n Merge returns a sorted list containing all elements from x and y .

Proof.

We will proceed by induction on the total size of the lists being merged.

□ Base Case: ($n = 1$)

This will only occur if either x or y is empty, and the other list has exactly 1 element.

MergeSort - Correctness

Theorem

Merge correctly merges two sorted lists.

Theorem

Given two sorted lists, x and y , of total size n Merge returns a sorted list containing all elements from x and y .

Proof.

We will proceed by induction on the total size of the lists being merged.

□ Base Case: ($n = 1$)

This will only occur if either x or y is empty, and the other list has exactly 1 element.

■ Merge correctly merges the empty list with any other sorted list (steps 1 and 2).

MergeSort - Correctness

Theorem

Merge correctly merges two sorted lists.

Theorem

Given two sorted lists, x and y , of total size n Merge returns a sorted list containing all elements from x and y .

Proof.

We will proceed by induction on the total size of the lists being merged.

□ Base Case: ($n = 1$)

This will only occur if either x or y is empty, and the other list has exactly 1 element.

■ Merge correctly merges the empty list with any other sorted list (steps 1 and 2).

MergeSort - Correctness



MergeSort - Correctness

Proof (Cont.)

- Inductive Hypothesis: Suppose that, given two sorted lists, x and y , of total size h Merge returns a sorted list containing all elements from x and y .

MergeSort - Correctness

Proof (Cont.)

- Inductive Hypothesis: Suppose that, given two sorted lists, x and y , of total size h Merge returns a sorted list containing all elements from x and y .
- Inductive Step: Consider two sorted lists with total size $h + 1$.

MergeSort - Correctness

Proof (Cont.)

- Inductive Hypothesis: Suppose that, given two sorted lists, x and y , of total size h Merge returns a sorted list containing all elements from x and y .
- Inductive Step: Consider two sorted lists with total size $h + 1$.
 - In steps 3 and 4 of the algorithm, Merge correctly places the smallest element at the beginning of the list.

MergeSort - Correctness

Proof (Cont.)

- Inductive Hypothesis: Suppose that, given two sorted lists, x and y , of total size h Merge returns a sorted list containing all elements from x and y .
- Inductive Step: Consider two sorted lists with total size $h + 1$.
 - In steps 3 and 4 of the algorithm, Merge correctly places the smallest element at the beginning of the list.
 - Merge then concatenates that element with the Merge of the remaining elements of the two lists.

MergeSort - Correctness

Proof (Cont.)

- Inductive Hypothesis: Suppose that, given two sorted lists, x and y , of total size h Merge returns a sorted list containing all elements from x and y .
- Inductive Step: Consider two sorted lists with total size $h + 1$.
 - In steps 3 and 4 of the algorithm, Merge correctly places the smallest element at the beginning of the list.
 - Merge then concatenates that element with the Merge of the remaining elements of the two lists.
 - The total size of the remaining two lists is h .

MergeSort - Correctness

Proof (Cont.)

- Inductive Hypothesis: Suppose that, given two sorted lists, x and y , of total size h Merge returns a sorted list containing all elements from x and y .
- Inductive Step: Consider two sorted lists with total size $h + 1$.
 - In steps 3 and 4 of the algorithm, Merge correctly places the smallest element at the beginning of the list.
 - Merge then concatenates that element with the Merge of the remaining elements of the two lists.
 - The total size of the remaining two lists is h .
 - By the Inductive Hypothesis, Merge correctly merges the remainder.
- Conclusion: Therefore, by PMI, Merge correctly merges two sorted lists.



MergeSort - Running Time



MergeSort - Running Time

- What is the running time?

MergeSort - Running Time

- What is the running time?
 - ▣ $T(n) = 2T(n/2) + O(n)$

MergeSort - Running Time

□ What is the running time?

■ $T(n) = 2T(n/2) + O(n) = O(n \log n)$

Median Finding



Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

For example, suppose the list of numbers is 14, 2, 3, 2, 7.

Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

For example, suppose the list of numbers is 14, 2, 3, 2, 7.
The median is 3.

Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

For example, suppose the list of numbers is 14, 2, 3, 2, 7.

The median is 3.

What if the list is even?

Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

For example, suppose the list of numbers is 14, 2, 3, 2, 7.

The median is 3.

What if the list is even?

We choose the smaller of the two middle elements.

Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

For example, suppose the list of numbers is 14, 2, 3, 2, 7.

The median is 3.

What if the list is even?

We choose the smaller of the two middle elements.

Median Finding

Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

For example, suppose the list of numbers is 14, 2, 3, 2, 7.

The median is 3.

What if the list is even?

We choose the smaller of the two middle elements.

Median Finding

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

For example, suppose the list of numbers is 14, 2, 3, 2, 7.

The median is 3.

What if the list is even?

We choose the smaller of the two middle elements.

Median Finding

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return the median element.

Median Finding

Definition

The *median* of a list of numbers is its 50th percentile. Half the numbers are bigger than the median and half the numbers are smaller.

For example, suppose the list of numbers is 14, 2, 3, 2, 7.

The median is 3.

What if the list is even?

We choose the smaller of the two middle elements.

Median Finding

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: Return the median element.

Any ideas?

Selection



Selection

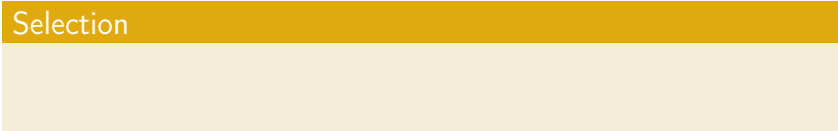


It is surprisingly easier to consider a more general problem, *selection*.

Selection



It is surprisingly easier to consider a more general problem, *selection*.



Selection

Selection

It is surprisingly easier to consider a more general problem, *selection*.

Selection

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$ and an integer k .

Selection

It is surprisingly easier to consider a more general problem, *selection*.

Selection

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$ and an integer k .

Goal: Return the k th smallest element of $a_1, a_2, a_3, \dots, a_n$.

Selection

It is surprisingly easier to consider a more general problem, *selection*.

Selection

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$ and an integer k .

Goal: Return the k th smallest element of $a_1, a_2, a_3, \dots, a_n$.

If $k = 1$, then the minimum element is returned.

Selection

It is surprisingly easier to consider a more general problem, *selection*.

Selection

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$ and an integer k .

Goal: Return the k th smallest element of $a_1, a_2, a_3, \dots, a_n$.

If $k = 1$, then the minimum element is returned.

If $k = n$, then the maximum element is returned.

Selection

It is surprisingly easier to consider a more general problem, *selection*.

Selection

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$ and an integer k .

Goal: Return the k th smallest element of $a_1, a_2, a_3, \dots, a_n$.

If $k = 1$, then the minimum element is returned.

If $k = n$, then the maximum element is returned.

If $k = \lfloor \frac{n}{2} \rfloor$, then the median is returned.

Randomized Selection



Randomized Selection

$\text{RandomSelection}(A[1, \dots, n], k)$

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

1: **if** $n \leq 1$ **then**

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

```
1: if  $n \leq 1$  then  
    return  $A[1]$ 
```

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

```
1: if  $n \leq 1$  then  
    return  $A[1]$   
2: else
```

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

1: **if** $n \leq 1$ **then**

return $A[1]$

2: **else**

3: Randomly choose an element from A , call it x .

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

1: **if** $n \leq 1$ **then**

return $A[1]$

2: **else**

3: Randomly choose an element from A , call it x .

4: Let A_L be the numbers in the list less than x , A_R be the numbers in the list greater than x , and A_x be the numbers in the list equal to x .

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

1: **if** $n \leq 1$ **then**

return $A[1]$

2: **else**

3: Randomly choose an element from A , call it x .

4: Let A_L be the numbers in the list less than x , A_R be the numbers in the list greater than x , and A_x be the numbers in the list equal to x .

5: **if** $k \leq |A_L|$ **then return** RandomSelection(A_L, k)

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

1: **if** $n \leq 1$ **then**

return $A[1]$

2: **else**

3: Randomly choose an element from A , call it x .

4: Let A_L be the numbers in the list less than x , A_R be the numbers in the list greater than x , and A_x be the numbers in the list equal to x .

5: **if** $k \leq |A_L|$ **then return** RandomSelection(A_L, k)

6: **if** $|A_L| < k \leq |A_L| + |A_x|$ **then return** x

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

1: **if** $n \leq 1$ **then**

return $A[1]$

2: **else**

3: Randomly choose an element from A , call it x .

4: Let A_L be the numbers in the list less than x , A_R be the numbers in the list greater than x , and A_x be the numbers in the list equal to x .

5: **if** $k \leq |A_L|$ **then return** RandomSelection(A_L, k)

6: **if** $|A_L| < k \leq |A_L| + |A_x|$ **then return** x

7: **else** **return** RandomSelection($A_R, k - |A_L| - |A_x|$)

Randomized Selection

RandomSelection($A[1, \dots, n], k$)

Input: A list of unsorted numbers $A[1, \dots, n]$ and an integer k .

Output: The k th smallest element of A .

1: **if** $n \leq 1$ **then**

return $A[1]$

2: **else**

3: Randomly choose an element from A , call it x .

4: Let A_L be the numbers in the list less than x , A_R be the numbers in the list greater than x , and A_x be the numbers in the list equal to x .

5: **if** $k \leq |A_L|$ **then return** RandomSelection(A_L, k)

6: **if** $|A_L| < k \leq |A_L| + |A_x|$ **then return** x

7: **else** **return** RandomSelection($A_R, k - |A_L| - |A_x|$)

RandomSelection



What is the running time?

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time.

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time. If we could choose x so that roughly half of the elements in the list are in A_L and the other half are in A_R , then our running time would be:

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time. If we could choose x so that roughly half of the elements in the list are in A_L and the other half are in A_R , then our running time would be:

$$T(n) = T(n/2) + O(n) = O(n)$$

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time. If we could choose x so that roughly half of the elements in the list are in A_L and the other half are in A_R , then our running time would be:

$$T(n) = T(n/2) + O(n) = O(n)$$

But that would only work if x is the median!

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time. If we could choose x so that roughly half of the elements in the list are in A_L and the other half are in A_R , then our running time would be:

$$T(n) = T(n/2) + O(n) = O(n)$$

But that would only work if x is the median!

□ Worst Case?

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time. If we could choose x so that roughly half of the elements in the list are in A_L and the other half are in A_R , then our running time would be:

$$T(n) = T(n/2) + O(n) = O(n)$$

But that would only work if x is the median!

□ Worst Case?

- In the worst case, we may select x to be the largest or smallest element over and over - that would only shrink our list by one at each iteration:

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time. If we could choose x so that roughly half of the elements in the list are in A_L and the other half are in A_R , then our running time would be:

$$T(n) = T(n/2) + O(n) = O(n)$$

But that would only work if x is the median!

□ Worst Case?

■ In the worst case, we may select x to be the largest or smallest element over and over - that would only shrink our list by one at each iteration:

$$T(n) = T(n - 1) + O(n) = O(n^2)$$

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time. If we could choose x so that roughly half of the elements in the list are in A_L and the other half are in A_R , then our running time would be:

$$T(n) = T(n/2) + O(n) = O(n)$$

But that would only work if x is the median!

□ Worst Case?

■ In the worst case, we may select x to be the largest or smallest element over and over - that would only shrink our list by one at each iteration:

$$T(n) = T(n - 1) + O(n) = O(n^2)$$

Luckily it turns out this is highly unlikely.

RandomSelection

What is the running time?

We can build, A_L , A_R , and A_x in linear time. If we could choose x so that roughly half of the elements in the list are in A_L and the other half are in A_R , then our running time would be:

$$T(n) = T(n/2) + O(n) = O(n)$$

But that would only work if x is the median!

□ Worst Case?

- In the worst case, we may select x to be the largest or smallest element over and over - that would only shrink our list by one at each iteration:

$$T(n) = T(n - 1) + O(n) = O(n^2)$$

Luckily it turns out this is highly unlikely.

□ Average Case?

RandomSelection Average Case Running Time



RandomSelection Average Case Running Time

Let's call a choice of x "good" if it lies within the 25th to 75th percentile.

RandomSelection Average Case Running Time

Let's call a choice of x "good" if it lies within the 25th to 75th percentile. Then:

$$|A_L| \leq 3/4|A| \text{ and } |A_R| \leq 3/4|A|$$

RandomSelection Average Case Running Time

Let's call a choice of x "good" if it lies within the 25th to 75th percentile. Then:

$$|A_L| \leq 3/4|A| \text{ and } |A_R| \leq 3/4|A|$$

How many x values do we have to pick (on average) before a good one is found?

RandomSelection Average Case Running Time

Let's call a choice of x "good" if it lies within the 25th to 75th percentile. Then:

$$|A_L| \leq 3/4|A| \text{ and } |A_R| \leq 3/4|A|$$

How many x values do we have to pick (on average) before a good one is found?

Lemma

On average a fair coin needs to be tossed twice before a "heads" is seen.

RandomSelection Average Case Running Time

Let's call a choice of x "good" if it lies within the 25th to 75th percentile. Then:

$$|A_L| \leq 3/4|A| \text{ and } |A_R| \leq 3/4|A|$$

How many x values do we have to pick (on average) before a good one is found?

Lemma

On average a fair coin needs to be tossed twice before a "heads" is seen.

The proof hinges on the fact that if E is the expected number of tosses before a heads is seen, $E = 1 + \frac{1}{2}E$.

RandomSelection Average Case Running Time

Let's call a choice of x "good" if it lies within the 25th to 75th percentile. Then:

$$|A_L| \leq 3/4|A| \text{ and } |A_R| \leq 3/4|A|$$

How many x values do we have to pick (on average) before a good one is found?

Lemma

On average a fair coin needs to be tossed twice before a "heads" is seen.

The proof hinges on the fact that if E is the expected number of tosses before a heads is seen, $E = 1 + \frac{1}{2}E$.

Therefore, on average, after two choices of x , the array will be reduced to at most $3/4$ its original size.

RandomSelection Average Case Running Time

Let's call a choice of x "good" if it lies within the 25th to 75th percentile. Then:

$$|A_L| \leq 3/4|A| \text{ and } |A_R| \leq 3/4|A|$$

How many x values do we have to pick (on average) before a good one is found?

Lemma

On average a fair coin needs to be tossed twice before a "heads" is seen.

The proof hinges on the fact that if E is the expected number of tosses before a heads is seen, $E = 1 + \frac{1}{2}E$.

Therefore, on average, after two choices of x , the array will be reduced to at most $3/4$ its original size.

The *expected* running time is:

RandomSelection Average Case Running Time

Let's call a choice of x "good" if it lies within the 25th to 75th percentile. Then:

$$|A_L| \leq 3/4|A| \text{ and } |A_R| \leq 3/4|A|$$

How many x values do we have to pick (on average) before a good one is found?

Lemma

On average a fair coin needs to be tossed twice before a "heads" is seen.

The proof hinges on the fact that if E is the expected number of tosses before a heads is seen, $E = 1 + \frac{1}{2}E$.

Therefore, on average, after two choices of x , the array will be reduced to at most $3/4$ its original size.

The *expected* running time is:

$$T(n) \leq T(3n/4) + O(n) = O(n)$$

RandomSelection Correctness



RandomSelection Correctness



With randomized algorithms, it's important to check that the algorithm terminates.

RandomSelection Correctness

With randomized algorithms, it's important to check that the algorithm terminates.

Does RandomSelection terminate?

RandomSelection Correctness

With randomized algorithms, it's important to check that the algorithm terminates.

Does RandomSelection terminate?

Lemma

Given a list, A , of size n and a value, k , RandomSelection(A, k) correctly finds the k th smallest element of A .

RandomSelection Correctness

With randomized algorithms, it's important to check that the algorithm terminates.

Does RandomSelection terminate?

Lemma

Given a list, A , of size n and a value, k , RandomSelection(A, k) correctly finds the k th smallest element of A .

Proof.

Use induction over n .



In Class Exercise: Peaks



In Class Exercise: Peaks

Peaks

Suppose you are given a list, A , with n entries, each entry holding a distinct number. You are told that the sequence of values $A[1], A[2], \dots, A[n]$ is *unimodal*: For some index, p , between 1 and n , the values of the list increase up to position p and decrease until position n .

In Class Exercise: Peaks

Peaks

Suppose you are given a list, A , with n entries, each entry holding a distinct number. You are told that the sequence of values $A[1], A[2], \dots, A[n]$ is *unimodal*: For some index, p , between 1 and n , the values of the list increase up to position p and decrease until position n .

- Find the "peak" entry.

In Class Exercise: Peaks

Peaks

Suppose you are given a list, A , with n entries, each entry holding a distinct number. You are told that the sequence of values $A[1], A[2], \dots, A[n]$ is *unimodal*: For some index, p , between 1 and n , the values of the list increase up to position p and decrease until position n .

- Find the "peak" entry.
- Is your algorithm correct?

In Class Exercise: Peaks

Peaks

Suppose you are given a list, A , with n entries, each entry holding a distinct number. You are told that the sequence of values $A[1], A[2], \dots, A[n]$ is *unimodal*: For some index, p , between 1 and n , the values of the list increase up to position p and decrease until position n .

- Find the "peak" entry.
- Is your algorithm correct?
- What is the running time?

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

1: **if** $|A| = 1$ **then**

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

```
1: if  $|A| = 1$  then  
    return  $A[1]$ 
```

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
- 2: **if** $|A| = 2$ **then**

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
- 2: **if** $|A| = 2$ **then**
 return $\max\{A[1], A[2]\}$

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
- 2: **if** $|A| = 2$ **then**
 return $\max\{A[1], A[2]\}$
- 3: $mid = \lceil n/2 \rceil$

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
- 2: **if** $|A| = 2$ **then**
 return $\max\{A[1], A[2]\}$
- 3: $mid = \lceil n/2 \rceil$
- 4: **if** $A[mid] > A[mid - 1]$ AND $A[mid] > A[mid + 1]$ **then**

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
- 2: **if** $|A| = 2$ **then**
 return $\max\{A[1], A[2]\}$
- 3: $mid = \lceil n/2 \rceil$
- 4: **if** $A[mid] > A[mid - 1]$ AND $A[mid] > A[mid + 1]$ **then**
 return $A[mid]$

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
- 2: **if** $|A| = 2$ **then**
 return $\max\{A[1], A[2]\}$
- 3: $mid = \lceil n/2 \rceil$
- 4: **if** $A[mid] > A[mid - 1]$ AND $A[mid] > A[mid + 1]$ **then**
 return $A[mid]$
- 5: **else if** $A[mid] > A[mid - 1]$ AND $A[mid] < A[mid + 1]$ **then**

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
- 2: **if** $|A| = 2$ **then**
 return $\max\{A[1], A[2]\}$
- 3: $mid = \lceil n/2 \rceil$
- 4: **if** $A[mid] > A[mid - 1]$ **AND** $A[mid] > A[mid + 1]$ **then**
 return $A[mid]$
- 5: **else if** $A[mid] > A[mid - 1]$ **AND** $A[mid] < A[mid + 1]$ **then**
 return FindPeaks($A[mid, \dots n]$)

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
- 2: **if** $|A| = 2$ **then**
 return $\max\{A[1], A[2]\}$
- 3: $mid = \lceil n/2 \rceil$
- 4: **if** $A[mid] > A[mid - 1]$ **AND** $A[mid] > A[mid + 1]$ **then**
 return $A[mid]$
- 5: **else if** $A[mid] > A[mid - 1]$ **AND** $A[mid] < A[mid + 1]$ **then**
 return FindPeaks($A[mid, \dots, n]$)
- 6: **else if** $A[mid] < A[mid - 1]$ **AND** $A[mid] > A[mid + 1]$ **then**

Peaks - Pseudocode

FindPeaks($A[1, \dots, n]$)

Input: A unimodal list of distinct numbers $A[1, \dots, n]$.

Output: The peak entry.

- 1: **if** $|A| = 1$ **then**
 return $A[1]$
 - 2: **if** $|A| = 2$ **then**
 return $\max\{A[1], A[2]\}$
 - 3: $mid = \lceil n/2 \rceil$
 - 4: **if** $A[mid] > A[mid - 1]$ **AND** $A[mid] > A[mid + 1]$ **then**
 return $A[mid]$
 - 5: **else if** $A[mid] > A[mid - 1]$ **AND** $A[mid] < A[mid + 1]$ **then**
 return FindPeaks($A[mid, \dots, n]$)
 - 6: **else if** $A[mid] < A[mid - 1]$ **AND** $A[mid] > A[mid + 1]$ **then**
 return FindPeaks($A[1, \dots, mid]$)
-

Peaks - Running Time



Peaks - Running Time

- We reduce a problem of size n to a single problem of size $n/2$.

Peaks - Running Time

- We reduce a problem of size n to a single problem of size $n/2$.
- There are a constant number of comparisons at each level of recursion.

Peaks - Running Time

- We reduce a problem of size n to a single problem of size $n/2$.
- There are a constant number of comparisons at each level of recursion.
- $T(n) = T(n/2) + O(1)$

Peaks - Running Time

- We reduce a problem of size n to a single problem of size $n/2$.
- There are a constant number of comparisons at each level of recursion.
- $T(n) = T(n/2) + O(1) = O(\log_2 n)$

Peaks - Correctness



Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof.

We proceed by strong induction.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof.

We proceed by strong induction.

□ Base Cases:

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof.

We proceed by strong induction.

□ Base Cases:

- If there is one element (as in part 1 of FindPeaks), then FindPeaks correctly returns that element.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof.

We proceed by strong induction.

□ Base Cases:

- If there is one element (as in part 1 of FindPeaks), then FindPeaks correctly returns that element.
- If there are two elements (as in part 2 of FindPeaks), then the larger of the two is the peak and FindPeaks correctly returns it.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof.

We proceed by strong induction.

- Base Cases:

- If there is one element (as in part 1 of FindPeaks), then FindPeaks correctly returns that element.
- If there are two elements (as in part 2 of FindPeaks), then the larger of the two is the peak and FindPeaks correctly returns it.

- Induction Hypothesis:

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof.

We proceed by strong induction.

□ Base Cases:

- If there is one element (as in part 1 of FindPeaks), then FindPeaks correctly returns that element.
- If there are two elements (as in part 2 of FindPeaks), then the larger of the two is the peak and FindPeaks correctly returns it.

□ Induction Hypothesis:

- Suppose that FindPeaks correctly finds the peak in a unimodal list of less than or equal to k distinct numbers.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof (Cont.)

- Consider a unimodal list of $k + 1$ distinct numbers.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof (Cont.)

- Consider a unimodal list of $k + 1$ distinct numbers.
 - If the midpoint ($mid = \lceil (k + 1)/2 \rceil$) is the peak, then $A[mid] > A[mid - 1]$ and $A[mid] > A[mid + 1]$. FindPeaks will correctly return $A[mid]$.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof (Cont.)

- Consider a unimodal list of $k + 1$ distinct numbers.
 - If the midpoint ($mid = \lceil (k + 1)/2 \rceil$) is the peak, then $A[mid] > A[mid - 1]$ and $A[mid] > A[mid + 1]$. FindPeaks will correctly return $A[mid]$.
 - If mid is among the increasing portion of the list ($A[mid] > A[mid - 1]$ and $A[mid] < A[mid + 1]$), then the peak is in the second half of the list ($A[mid, \dots, k]$).

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof (Cont.)

- Consider a unimodal list of $k + 1$ distinct numbers.
 - If the midpoint ($mid = \lceil (k + 1)/2 \rceil$) is the peak, then $A[mid] > A[mid - 1]$ and $A[mid] > A[mid + 1]$. FindPeaks will correctly return $A[mid]$.
 - If mid is among the increasing portion of the list ($A[mid] > A[mid - 1]$ and $A[mid] < A[mid + 1]$), then the peak is in the second half of the list ($A[mid, \dots, k]$). FindPeaks returns $\text{FindPeaks}(A[mid, \dots, n])$, which, by the induction hypothesis correctly finds the peak because the list has size less than or equal to k .

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof (Cont.)

- ■ If mid is among the decreasing portion of the list ($A[mid] < A[mid - 1]$ and $A[mid] > A[mid + 1]$), then the peak is in the first half of the list ($A[1, \dots, mid]$).

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof (Cont.)

- ■ If mid is among the decreasing portion of the list ($A[mid] < A[mid - 1]$ and $A[mid] > A[mid + 1]$), then the peak is in the first half of the list ($A[1, \dots, mid]$). FindPeaks returns $\text{FindPeaks}(A[1, \dots, mid])$, which, by the induction hypothesis correctly finds the peak because the list has size less than or equal to n .

Peaks - Correctness

Theorem

FindPeaks correctly finds the peak in a unimodal list of n distinct numbers.

Proof (Cont.)

- ■ If mid is among the decreasing portion of the list ($A[mid] < A[mid - 1]$ and $A[mid] > A[mid + 1]$), then the peak is in the first half of the list ($A[1, \dots, mid]$). FindPeaks returns $\text{FindPeaks}(A[1, \dots, mid])$, which, by the induction hypothesis correctly finds the peak because the list has size less than or equal to n .
- Therefore, by the principle of mathematical induction, we have the result.

